

PROGRAMACIÓN ORIENTADA A OBJETOS

Guía de ejercicios prácticos

Conceptos cubiertos: Encapsulamiento · Herencia · Abstracción · Polimorfismo

Instrucciones generales

Cada ejercicio incluye un enunciado, un fragmento de código de partida con marcadores TODO y una pista de solución. Los fragmentos de código están escritos para Unity con C# y asumen conocimiento básico de MonoBehaviour.

Se recomienda crear un proyecto Unity nuevo y un script por ejercicio. No es necesario completar todos los TODO para considerar el ejercicio resuelto: la jerarquía de clases y el razonamiento sobre modificadores de acceso son el objetivo principal.

1 | El jugador que nunca puede morir de mas

Encapsulamiento

Objetivo

Proteger el atributo vida usando private y garantizar que nunca sea menor a cero mediante métodos de acceso controlado.

Enunciado

Crea una clase Jugador que herede de MonoBehaviour. La variable vida debe ser privada. Implementa el método público RecibirDanio(int cantidad) que valide el límite inferior, y una propiedad de solo lectura VidaActual. Agrega también el método Curar(int cantidad) que no permita superar la vidaMaxima.

Código de partida

```
public class Jugador : MonoBehaviour
{
    [SerializeField] private int vidaMaxima = 100;
    private int _vida;

    public int VidaActual => _vida;

    void Start() => _vida = vidaMaxima;

    public void RecibirDanio(int cantidad)
    {
        // TODO: _vida nunca debe ser < 0
        // TODO: si _vida == 0, llamar Morir()
    }
}
```

```

    public void Curar(int cantidad)
    {
        // TODO: _vida nunca debe superar vidaMaxima
    }

    private void Morir()
    {
        // TODO: desactivar el GameObject
    }
}

```

Guía de solución

Usa `Mathf.Max(0, _vida - cantidad)` para el dano y `Mathf.Min(vidaMaxima, _vida + cantidad)` para la cura. El atributo `[SerializeField]` permite editar `vidaMaxima` en el Inspector sin romper el encapsulamiento.

2

La familia de ítems coleccionables

Herencia

Objetivo

Crear una jerarquía de clases reutilizando atributos y comportamiento común sin duplicar código.

Enunciado

Diseña la clase base Ítem con los atributos nombre y valor, y el método Recolectar(). Luego crea tres subclases: Moneda (suma puntaje), Llave (abre puertas) y Poción (restaura vida). Cada subclase debe llamar al comportamiento base con `base.Recolectar()` y agregar su propio comportamiento.

Código de partida

```

public class Item : MonoBehaviour
{
    protected string nombre;
    protected int valor;

    public virtual void Recolectar(Jugador jugador)
    {
        Debug.Log($"Recogiste: {nombre}");
        Destroy(gameObject);
    }
}

public class Moneda : Item
{
    void Awake() { nombre = "Moneda de oro"; valor = 10; }

    public override void Recolectar(Jugador jugador)
    {
        base.Recolectar(jugador);
    }
}

```

```

    {
        // TODO: suma valor al puntaje del jugador
        // TODO: llama a base.Recolectar(jugador)
    }
}

// TODO: implementar Llave y Poción de forma similar

```

Guía de solución

base.Recolectar(jugador) ejecuta el comportamiento compartido (Debug.Log + Destroy). Cada subclase lo extiende con override.

3 | Enemigos con jerarquía propia

Herencia

Objetivo

Modelar distintos tipos de enemigos con comportamiento compartido y diferenciado usando herencia de MonoBehaviour.

Enunciado

Crea la superclase Enemigo con vida, velocidad y el método virtual Move(). Implementa tres subclases: Goomba (camina en línea recta), Koopa (invierte la dirección al chocar) y Lakitu (flota y lanza Spinies). Cada subclase inicializa sus stats en Start() y sobrescribe Move() cuando corresponde.

Código de partida

```

public class Enemigo : MonoBehaviour
{
    protected float vida;
    protected float velocidad;

    public virtual void Move()
    {
        transform.Translate(Vector3.left * velocidad * Time.deltaTime);
    }

    public void RecibirGolpe(float dano)
    {
        vida -= dano;
        if (vida <= 0) Destroy(gameObject);
    }
}

```

```
public class Goomba : Enemigo
{
    void Start() { vida = 1f; velocidad = 2f; }
    // Hereda Move() sin cambios
}

// TODO: Koopa con cambio de dirección al chocar
// TODO: Lakitu con movimiento vertical + Instantiate de Spinies
```

Guía de solución

Koopa puede usar un bool yendoDerecha y cambiar entre Vector3.left y Vector3.right en OnCollisionEnter2D. Lakitu puede usar InvokeRepeating("LanzarSpiny", 0f, 3f) en su Start().

4 | Ataques polimórfico

Polimorfismo

Objetivo

Lograr que un GameManager llame Atacar() sobre cualquier enemigo sin conocer su tipo concreto en tiempo de compilación.

Enunciado

Agrega el método public virtual void Atacar() a la clase Enemigo. Implementa tres versiones con override: Planta (mordisco en área cercana con OverlapCircle), Bill (proyectil con Instantiate) y Bowser (fuego + salto). El GameManager itera un array Enemigo[] y llama Atacar() en cada elemento sin ningún if y switch.

Código de partida

```
// Enemigo.cs
public virtual void Atacar()
{
    Debug.Log($"{name} usa ataque base");
}

// Planta.cs
public override void Atacar()
{
    // TODO: Physics2D.OverlapCircle para dano en area
}

// Bill.cs
public override void Atacar()
{
    // TODO: Instantiate(proyectilPrefab, transform.position, transform.rotation)
}
```

```
// GameManager.cs
private Enemy[] _enemigos;

void Update()
{
    foreach (Enemy e in _enemigos)
        e.Atacar();
}
```

Guía de solución

El array es de tipo Enemy[] pero en tiempo de ejecución Unity invoca el override correcto según el tipo real. Agregar Bowser no requiere modificar GameManager: solo crear la nueva clase con su propio override.

5

Sistema de estados con encapsulamiento

Encapsulamiento

Objetivo

Implementar una máquina de estados donde las transiciones están controladas y validadas internamente, sin exposición directa del estado.

Enunciado

Crea la clase EstadoJugador con un enum privado (Idle, Corriendo, Saltando, Muerto). El estado sólo puede cambiar mediante métodos públicos (Correr, Saltar, Morir, Detenerse) que validen si la transición es legal. Desde afuera solo se puede leer el estado actual mediante una propiedad.

Código de partida

```
public class EstadoJugador : MonoBehaviour
{
    private enum Estado { Idle, Corriendo, Saltando, Muerto }

    private Estado _estadoActual = Estado.Idle;
    public string EstadoActual => _estadoActual.ToString();

    public void Correr()
    {
        // TODO: solo valido desde Idle
    }

    public void Saltar()
    {
        // TODO: válido desde Idle o Corriendo
    }
}
```

```

    }

    public void Morir()
    {
        // TODO: válido desde cualquier estado excepto Muerto
    }

    private void CambiarEstado(Estado nuevo)
    {
        Debug.Log($"Transicion: {_estadoActual} -> {nuevo}");
        _estadoActual = nuevo;
    }
}

```

Guia de solucion

Usa guard clauses al inicio de cada método: `if (_estadoActual == Estado.Muerto) return;` El código externo puede pedir transiciones pero no puede asignar el estado directamente: eso es encapsulamiento aplicado a lógica de negocio.

6 | Armas con clase base abstracta

Abstracción

Objetivo

Definir el contrato común de todas las armas mediante clase abstracta, obligando a cada implementación a definir su propia lógica de disparo.

Enunciado

Crea la clase abstracta Arma con daño, cooldown y el método abstracto Disparar(). Agrega PuedoDisparar() que verifique el cooldown con Time.time. Implementa tres armas: Pistola (un proyectil), Escopeta (rafaga de tres en abanico) y Laser (Raycast continuo).

Código de partida

```

public abstract class Arma : MonoBehaviour
{
    protected float dano;
    protected float cooldown;
    private float _tiempoUltimoDisparo;

    public abstract void Disparar();

    protected bool PuedoDisparar()

```

```

        => Time.time >= _tiempoUltimoDisparo + cooldown;

        protected void RegistrarDisparo()
            => _tiempoUltimoDisparo = Time.time;
    }

    public class Pistola : Arma
    {
        public GameObject proyectilPrefab;

        void Start() { dano = 10f; cooldown = 0.5f; }

        public override void Disparar()
        {
            if (!PuedoDisparar()) return;
            // TODO: Instantiate del proyectil
            // TODO: RegistrarDisparo()
        }
    }

    // TODO: Escopeta -> 3 proyectiles en angulos distintos
    // TODO: Laser -> Physics2D.Raycast

```

Guia de solucion

Para Escopeta, itera 3 veces y aplica `Quaternion.Euler(0, 0, angulo)` a cada proyectil (-15, 0, 15 grados). Para Laser, usa `Debug.DrawRay` en el editor y `RaycastHit2D hit = Physics2D.Raycast(...)` para aplicar el dano.

Proyecto integrador: Dungeon

Encapsulamiento · Herencia · Abstracción · Polimorfismo

Objetivo

Aplicar los cuatro pilares de POO en un único sistema de dungeon crawler. Este ejercicio no tiene una sola solución correcta: se evalúa la coherencia del diseño.

Enunciado

Diseña el esqueleto completo: (1) Personaje base abstracto con vida encapsulada y método abstracto Acción(). (2) Clases intermedias Héroe y Monstruo que heredan de Personaje. (3) Al menos dos subclases concretas de cada una: Guerrero y Mago, Esqueleto y Dragon. (4) Un Dungeon que itere sobre Personaje[] y ejecute Acción() sin ninguna condición sobre el tipo.

Código de partida

```
public abstract class Personaje : MonoBehaviour
{
    [SerializeField] private int _vida;
    public int Vida => _vida;

    public abstract void Accion();
    public virtual void RecibirDanio(int d) { _vida -= d; }
}

public abstract class Heroe : Personaje
{
    protected int poderAtaque;
    // Accion() sigue abstracto para Guerrero y Mago
}

public class Guerrero : Heroe
{
    void Start() { poderAtaque = 15; }
    public override void Accion()
    {
        // TODO: ataque fisico con animacion
    }
}

// TODO: Mago : Heroe      -> lanza hechizos
// TODO: Esqueleto : Monstruo -> ataque basico
// TODO: Dragon : Monstruo  -> fuego + vuelo

public class Dungeon : MonoBehaviour
{
    private Personaje[] _personajes;
    void Start() => _personajes = FindObjectsOfType<Personaje>();
    void Update() { foreach (var p in _personajes) p.Accion(); }
}
```

Guia de solucion

La cadena de abstraccion es: Personaje (abstracto) -> Heroe (abstracto) -> Guerrero (concreto). El Dungeon funciona con cualquier personaje sin un solo if o switch. Agregar Dragon o Paladin es crear una nueva clase, no modificar ninguna existente.